

RefSyn の手動解析記録

1 対象関数: append

現在のノードを先頭とする連結リストの末尾に、新しいノードを追加するメソッド。引数として渡された値を `val` フィールドにもつノードを新たに作成し、それをリストの最後に接続。

1.1 初期プログラム

```
class Node {  
  append(arg) { /* TBD */ }  
}  
var lst = new Node();  
lst.val = 2;  
lst.append(0);  
lst.append(3);
```

1.2 操作ログ

1.2.1 `lst.append(0)` に対する操作

- `addNode: __temp1, Node, false, undefined`
- `addNode: __temp2, 0, true, string`
- `addEdge: __temp1, __temp2, val`
- `addEdge: main-new1, __temp1, next`

1.2.2 `lst.append(3)` に対する操作

- `addNode: __temp3, Node, false, undefined`
- `addNode: __temp4, 3, true, string`
- `addEdge: __temp3, __temp4, val`
- `addEdge: temp1, __temp3, next`

1.3 手動で復元したコード

1.3.1 lst.append(0)

```
var temp1 = new Node();
var temp2 = 0;
temp1.val = temp2;
this.next = temp1;
```

1.3.2 lst.append(3)

```
var temp1 = new Node();
var temp2 = 3;
temp1.val = temp2;
this.next.next = temp1;
```

1.4 一般化と部分関数の推定

```
var temp1 = new Node();
var temp2 = f();    // f() = 引数 arg を返す関数
temp1.val = temp2;
g().next = temp1;   // g() = 現在のリストの末尾を返す関数
```

1.5 想定の合成された関数

```
append(arg) {
  var temp1 = new Node();
  function f() {
    return arg;
  }
  var temp2 = f();
  temp1.val = temp2;
  function g() {
    if (!this.next) {
      return this;
    } else {
      this.next.g();    // できないけど
    }
  }
  g().next = temp1;
}
```

1.6 oracle の関数

```
append(arg) {
  if(!this.next) {
    this.next = new Node();
    this.net.val = arg;
  } else {
    this.next.append(arg);
  }
}
```

2 対象関数: prepend

現在のノードを先頭とする連結リストの先頭に、新しいノードを追加するメソッド。引数として渡された値を val フィールドにもつノードを新たに作成し、それをリストの先頭に接続。

2.1 初期プログラム

```
class Node {
  prepend(arg) { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst = lst.prepend(0);
lst = lst.prepend(3);
```

2.2 操作ログ

2.2.1 lst.prepend(0) に対する操作

- addNode: __temp1, Node, false, undefined
- addNode: __temp2, 0, true, string
- addEdge: __temp1, __temp2, val
- addEdge: __temp1, main-new1, next
- addVariable: __temp1, return

2.2.2 lst.prepend(3) に対する操作

- addNode: __temp3, Node, false, undefined
- addNode: __temp4, 3, true, string

- addEdge: __temp3, __temp4, val
- addEdge: __temp3, __temp1, next
- addVariable: __temp3, return

2.3 手動で復元したコード

2.3.1 lst.prepend(0)

```
var temp1 = new Node();
var temp2 = 0;
temp1.val = temp2;
temp1.next = this;
return temp1;
```

2.3.2 lst.prepend(3)

```
var temp3 = new Node();
var temp4 = 3;
temp3.val = temp4;
temp3.next = this;
return temp3;
```

temp1 を this にすることは実行時情報から判断できる？

2.4 一般化と部分関数の推定

```
var temp1 = new Node();
var temp2 = f(); // f() = 引数 arg を返す関数
temp1.val = temp2;
temp1.next = this;
return temp1;
```

2.5 想定合成された関数

```
prepend(arg) {
  var temp1 = new Node();
  function f() {
    return arg;
  }
  var temp2 = f();
  temp1.val = temp2;
  temp1.next = this;
  return temp1;
}
```

2.6 oracle の関数

```
prepend(arg) {  
  var temp1 = new Node();  
  temp1.val = arg;  
  temp1.next = this;  
  return temp1;  
}
```

3 対象関数: removeLast

現在のノードを先頭とする連結リストの最後尾のノードを消すメソッド。

3.1 初期プログラム

```
class Node {  
  removeLast() { /* TBD */ }  
}  
  
var lst = new Node();  
lst.val = 2;  
lst.next = new Node();  
lst.next.val = 0;  
lst.next.next = new Node();  
lst.next.next.val = 3;  
lst.removeLast();  
lst.removeLast();
```

3.2 操作ログ

3.2.1 lst.removeLast() に対する操作

- deleteNode: main-new3

3.2.2 lst.removeLast() に対する操作

- deleteNode: main-new2

next とか val とかも消しても良い
Node だけ消せば可視化されない

3.3 手動で復元したコード

3.3.1 lst.removeLast()

```
this.next.next = undefined
```

3.3.2 lst.removeLast()

```
this.next = undefined
```

next 先の Node が消えた
null ではなく、undefined
constructor があるなら null でよい

3.4 一般化と部分関数の推定

```
f().next = undefined;    // f() = 現在のリストの末尾を返す関数
```

3.5 想定合成された関数

```
removeLast() {  
  function f() {  
    if(!this.next.next) {  
      return this.next;  
    } else {  
      this.next.g();    // できないけど  
    }  
  }  
  f().next = undefined;  
}
```

3.6 oracle の関数

```
removeLast() {  
  if(this.next.next === undefined) {  
    this.next = undefined;  
  } else {  
    this.next.removeLast();  
  }  
}
```

4 対象関数: removeFirst

現在のノードを先頭とする連結リストの先頭のノードを消すメソッド。

4.1 初期プログラム

```

class Node {
    removeFirst() { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.next.next = new Node();
lst.next.next.val = 3;
lst = lst.removeFirst();
lst = lst.removeFirst();

```

4.2 操作ログ

4.2.1 lst.removeFirst() に対する操作

- addVariable: main-new2, return

4.2.2 lst.removeFirst() に対する操作

- addVariable: main-new3, return

4.3 手動で復元したコード

4.3.1 lst.prepend(0)

```
return this.next;
```

ここの変換は非自明？

4.3.2 lst.prepend(3)

```
return this.next;
```

4.4 一般化と部分関数の推定

```
return this.next;
```

4.5 想定の合成された関数

```

removeFirst() {
    return this.next;
}

```

4.6 oracle の関数

```
removeFirst() {  
    return this.next;  
}
```

5 対象関数: removeVal(arg)

連結リスト内の val フィールドが arg の値を持つノードを削除するメソッド。

5.1 初期プログラム

```
class Node {  
    removeVal(arg) { /* TBD */ }  
}  
  
var lst = new Node();  
lst.val = 2;  
lst.next = new Node();  
lst.next.val = 0;  
lst.next.next = new Node();  
lst.next.next.val = 3;  
lst.next.next.next = new Node();  
lst.next.next.next.val = 1;  
lst = lst.removeVal(0);  
lst = lst.removeVal(1);
```

5.2 操作ログ

5.2.1 lst.removeVal(0) に対する操作

- editEdgeReference: main-new1, main-new2, main-new3, next
- addVariable: main-new1, return

5.2.2 lst.removeVal(1) に対する操作

- deleteEdge: main-new3, main-new4, next
- addVariable: main-new1, return

5.3 手動で復元したコード

5.3.1 lst.removeVal(0)

```
this.next = this.next.next;  
return this;
```

5.3.2 lst.removeVal(1)

```
this.next.next = null;  
return this;
```

5.4 一般化と部分関数の推定

```
f().next = g()  
// f は val フィールドに arg をもつノードの一個前  
// g は val フィールドに arg をもつノードの一個後
```

5.5 想定的合成された関数

```
removeVal(arg) {  
    f().next = g();  
    return this;  
    f() {  
        if(this.val == arg) {  
  
        }  
    }  
    g() {  
        if(this.val == arg) {  
  
        }  
    }  
}
```

5.6 oracle の関数

```
removeVal(arg) {  
    if(this.val == arg) {  
        return this.next;  
    }  
    if(this.next != null) {  
        this.next = this.next.removeval(arg);  
    }  
    return this;  
}
```

6 対象関数: removeAt(i)

連結リストの*i*番目のノードを削除するメソッド。

6.1 初期プログラム

```
class Node {
    removeAt(i) { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.next.next = new Node();
lst.next.next.val = 3;
lst.next.next.next = new Node();
lst.next.next.next.val = 1;
lst.removeAt(2);
lst.removeAt(1);
```

6.2 操作ログ

6.2.1 lst.removeAt(2) に対する操作

- editEdgeReference: main-new2, main-new3, main-new4, next
- addVariable: main-new1, return

6.2.2 lst.removeAt(1) に対する操作

- editEdgeReference: main-new1, main-new2, main-new4, next
- addVariable: main-new1, return

6.3 手動で復元したコード

6.3.1 lst.append(0)

```
this.next.next = this.next.next.next;
return this;
```

6.3.2 lst.append(3)

```
this.next = this.next.next;
return this;
```

6.4 一般化と部分関数の推定

```
f().next = g();
\\ f() = i-1 番目のノードを返す関数
\\ g() = i+1 番目のノードを返す関数
return this;
```

6.5 想定の合成された関数

```
removeAt(i) {
    f().next = g();
    return this;
    \\ f() = i-1 番目のノードを返す関数
    \\ g() = i+1 番目のノードを返す関数
}
```

6.6 oracle の関数

```
removeAt(i) {
    if(i=== 0) {
        return this.next;
    }
    this.next = this.next.remove(i - 1);
    return this;
}
```

7 対象関数: insertAfter(i, arg)

連結リストの*i*番目の後ろに、*val* フィールドに *arg* を持つ新しいノードを追加するメソッド。

7.1 初期プログラム

```
class Node {
    insertAfter(i, arg) { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.insertAfter(0,3);
lst.insertAfter(1,4);
```

7.2 操作ログ

7.2.1 `lst.insertAfter(0,3)` に対する操作

- `addNode: __temp1, Node, false, undefined`
- `addNode: __temp2, 3, true, string`
- `addEdge: __temp1, __temp2, val`
- `addEdge: __temp1, main-new2, next`
- `editEdgeReference: main-new1, main-new2, __temp1, next`

7.2.2 `lst.insertAftere(1,4)` に対する操作

- `addNode: __temp3, Node, false, undefined`
- `addNode: __temp4, 4, true, string`
- `addEdge: __temp3, __temp4, val`
- `addEdge: __temp3, main-new2, next`
- `editEdgeReference: __temp1, main-new2, __temp3, next`

7.3 手動で復元したコード

7.3.1 `lst.insertAfter(0,3)`

```
var temp1 = new Node();
var temp2 = 3;
temp1.val = temp2;
temp1.next = this.next;
this.next = temp1;
```

7.3.2 `lst.insetAfter(1,4)`

```
var temp1 = new Node();
var temp2 = 4;
temp1.val = temp2;
temp1.next = this.next.next;
this.next.next = temp1;
```

7.4 一般化と部分関数の推定

```
var temp1 = new Node();
var temp2 = f();          // f() = 引数 arg を返す関数
temp1.val = temp2;
temp1.next = g()          // g() = i+1 番目のノードを返す関数
h().next = temp1;        // h() = i-1 番目のノードを返す関数
```

7.5 想定の合成された関数

```
insertAfter(i, arg) {
  var temp1 = new Node();
  var temp2 = arg;
  temp1.val = temp2;
  temp1.next = find(i+1);
  find(i-1).next = temp1;
}
```

7.6 oracle の関数

```
insertAfter(i, arg) {
  function find(node, index) {
    if (node === null) {
      return null;
    }
    if (index === 0) {
      return node;
    }
    return find(node.next, index - 1);
  }
  const targetNode = find(this, i);
  if (targetNode !== null) {
    const newNode = new Node();
    newNode.val = arg;
    newNode.next = targetNode.next;
    targetNode.next = newNode;
  }
}
```

8 対象関数: concat(lst)

現在のノードを先頭とする連結リストの末尾に、別の連結リストを追加するメソッド。

8.1 初期プログラム

```
class Node {
    concat(otherList) { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.next.next = new Node();
lst.next.next.val = 3;
var list = new Node();
list.val = 1;
list.next = new Node();
list.next.val = 4;
var l = new Node();
l.val = 5;
lst.concat(list);
lst.concat(l);
```

8.2 操作ログ

8.2.1 lst.concat(list) に対する操作

- addEdge: main-new3, main-new4, next

8.2.2 lst.concat(l) に対する操作

- addNode: main-new5, main-new6, next

8.3 手動で復元したコード

8.3.1 lst.concat(list)

```
this.next.next = list;
```

8.3.2 lst.concat(l)

```
this.next.next.next.next.next = l;
```

8.4 一般化と部分関数の推定

```
f().next = g();
\\ f() = 最後のノードを返す関数
\\ g() = 引数を返す関数
```

8.5 想定の合成された関数

```
concat(otherList) {  
    f().next = otherList;  
    \\ f() = の最後のノードを返す関数this  
}
```

8.6 oracle の関数

```
concat(otherList) {  
    if(!this.next) {  
        this.next = otherList;  
    } else {  
        this.next.concat(otherList);  
    }  
}
```

9 対象関数: set(i, arg)

連結リストの i 番目の値を arg に変更するメソッド。

9.1 初期プログラム

```
class Node {  
    set(i, arg) { /* TBD */ }  
}  
  
var lst = new Node();  
lst.val = 2;  
lst.next = new Node();  
lst.next.val = 0;  
lst.next.next = new Node();  
lst.next.next.val = 3;  
lst.next.next.next = new Node();  
lst.next.next.next.val = 1;  
lst.set(2, 5);  
lst.set(1, 4);
```

9.2 操作ログ

9.2.1 lst.set(2, 5) に対する操作

- addNode: _temp1, 5, true, string
- editEdgeReference: main-new3, main-new3-val, _temp1, val

9.2.2 lst.set(1,4) に対する操作

- addNode: __temp2, 4, true, string
- addEdge: main-new2, main-new2-val, __temp2, val

9.3 手動で復元したコード

9.3.1 lst.set(2,5)

```
var temp1 = 5;
this.next.next.val = temp1;
```

9.3.2 lst.set(1,4)

```
var temp1 = 4;
this.next.val = temp2;
```

9.4 一般化と部分関数の推定

```
var temp1 = f();    \\ f() = 引数 arg を返す関数
g().val = temp1;    \\ g() = 番目のノードを返す関数i
```

9.5 想定的合成された関数

```
set(i, arg) {
  var temp1 = arg;
  find(i).val = temp1;
}
```

9.6 oracle の関数

```
set(i, arg) {
  if(i === 0) {
    this.val = arg;
  }
  if(i > 0) {
    this.next.set(i-1, arg);
  }
}
```

10 対象関数: swap(i, j)

現在のノードを先頭とする連結リストの i 番目と j 番目の要素を交換するメソッド。

10.1 初期プログラム

```
class Node {
    swap(i,j) { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.next.next = new Node();
lst.next.next.val = 3;
lst.next.next.next = new Node();
lst.next.next.next.val = 1;
lst.swap(0, 3);
lst.swap(1,2);
```

10.2 操作ログ

10.2.1 lst.swap(0,3) に対する操作

- addNode: __temp1, 2, true, string
- addNode: __temp2, 1, true, string
- editEdgeReference: main-new1, main-new1-val, __temp1, val
- editEdgeReference: main-new4, main-new4-val, __temp2, val

10.2.2 lst.swap(1,2) に対する操作

- addNode: __temp3, 3, true, string
- addNode: __temp4, 0, true, string
- editEdgeReference: main-new2, main-new2-val, __temp3, val
- editEdgeReference: main-new3, main-new3-val, __temp4, val

10.3 手動で復元したコード

操作列だけみると実際には違う挙動をするプログラムが出てくる
参照元がなくなったが使うものは先に temp とかで変数宣言しておく？

10.3.1 lst.swap(0,3)

```
var temp1 = 2;
var temp2 = 1;
this.val = temp1;
this.next.next.next.val = temp2;
```

10.3.2 lst.swap(1,2)

```
var temp1 = 3;
var temp2 = 0;
this.next.val = temp1;
this.next.next.val = temp2;
```

10.4 一般化と部分関数の推定

```
var temp1 = f();    \ \ f() = i 番目のノードの val フィールド
                     の値を返す関数
var temp2 = g();    \ \ g() = j 番目のノードの val フィールド
                     の値を返す関数
h().val = temp1;    \ \ h() = j 番目のノードを返す関数
i().val = temp2;    \ \ i() = i 番目のノードを返す関数
```

10.5 想定の合成された関数

```
swap(i,j) {
  var temp1 = getVal(i);
  var temp2 = getVal(j);
  getNode(j).val = temp1;
  getNode(i).val = temp2;
}
```

10.6 oracle の関数

```
swap(i,j) {
  function find(node, index) {
    if(index === 0) {
      return node;
    }
    return find(node.next, index - 1);
  }
  var nodeI = find(this, i);
  var nodeJ = find(this, j);
```

```

    var tempVal = nodeI.val;
    nodeI.val = nodeJ.val;
    nodeJ.val = tempVal;
}

```

11 対象関数: indexOf(arg)

連結リスト内で指定された値を持つノードのインデックスを返すメソッド。

11.1 初期プログラム

```

class Node {
  indexOf(arg) { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.next.next = new Node();
lst.next.next.val = 3;
var index0 = lst.indexOf(0);
var index3 = lst.indexOf(3);
var indexNotFound = lst.indexOf(99);

```

11.2 操作ログ

11.2.1 lst.indexOf(0) に対する操作

- addVariable: main-new2, return

11.2.2 lst.indexOf(3) に対する操作

- addVariable: main-new3, return

11.2.3 lst.indexOf(99) に対する操作

- addVariable: null, return

11.3 手動で復元したコード

11.3.1 lst.indexOf(0)

```

if (this.val === 0) return 0;
if (this.next.val === 0) return 1;
// ...

```

11.3.2 lst.indexOf(3)

```
if (this.val === 3) return 0;
if (this.next.val === 3) return 1;
if (this.next.next.val === 3) return 2;
// ...
```

11.4 一般化と部分関数の推定

```
// f() = 現在のノードからを探し、インデックスを返す関数 arg
return f();
```

11.5 想定 of 合成された関数

```
indexOf(arg) {
  let currentIndex = 0;
  let currentNode = this;
  while (currentNode !== null) {
    if (currentNode.val === arg) {
      return currentIndex;
    }
    currentNode = currentNode.next;
    currentIndex++;
  }
  return -1; // Not found
}
```

11.6 oracle の関数

```
indexOf(arg, current=this, index=0) {
  if (current === null) {
    return -1;
  }
  if (current.val === arg) {
    return index;
  }
  return this.indexOf(arg, current.next, index + 1);
}
```

12 対象関数: size()

連結リストのノード数を返すメソッド。

12.1 初期プログラム

```
class Node {
    size() { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.next.next = new Node();
lst.next.next.val = 3;
var s1 = lst.size();
var emptyLst = new Node();
var s2 = emptyLst.size();
```

12.2 操作ログ

12.2.1 lst.size() に対する操作

- addVariable: __temp1, 3, true, number
- addVariable: __temp1, return

12.2.2 emptyLst.size() に対する操作

- addVariable: __temp2, 1, true, number
- addVariable: __temp2, return

12.3 手動で復元したコード

12.3.1 lst.size()

```
var count = 0;
var current = this;
while (current !== null) {
    count++;
    current = current.next;
}
return count;
```

12.3.2 emptyLst.size()

```
return 1; // Assuming initial node counts as 1 even if val is undefined
```

12.4 一般化と部分関数の推定

```
// f() = リストを走査してノード数を数える関数
return f();
```

12.5 想定の合成された関数

```
size() {
  let count = 0;
  let current = this;
  while (current !== null) {
    count++;
    current = current.next;
  }
  return count;
}
```

12.6 oracle の関数

```
size(node = this) {
  if (node === null) {
    return 0;
  }
  return 1 + this.size(node.next);
}
```

13 対象関数: get(i)

連結リストの*i*番目のノードの `val` フィールドを返すメソッド。

13.1 初期プログラム

```
class Node {
  get(i) { /* TBD */ }
}

var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.next.next = new Node();
lst.next.next.val = 3;
var val0 = lst.get(0);
var val2 = lst.get(2);
```

13.2 操作ログ

13.2.1 `lst.get(0)` に対する操作

- `addVariable: main-new1-val, return`

13.2.2 `lst.get(2)` に対する操作

- `addVariable: main-new3-val, return`

13.3 手動で復元したコード

13.3.1 `lst.get(0)`

```
return this.val;
```

13.3.2 `lst.get(2)`

```
return this.next.next.val;
```

13.4 一般化と部分関数の推定

```
// f() = 番目のノードのフィールドを返す関数 ival  
return f();
```

13.5 想定的合成された関数

```
get(i) {  
  let currentIndex = 0;  
  let currentNode = this;  
  while (currentNode !== null && currentIndex < i) {  
    currentNode = currentNode.next;  
    currentIndex++;  
  }  
  if (currentNode !== null) {  
    return currentNode.val;  
  }  
  return undefined; // Or throw error  
}
```

13.6 oracle の関数

```
get(i, node = this) {
  if (node === null) {
    return undefined; // Or throw error
  }
  if (i === 0) {
    return node.val;
  }
  return this.get(i - 1, node.next);
}
```

14 対象関数: clear()

連結リストのすべてのノードを削除するメソッド。

14.1 初期プログラム

```
class Node {
  clear() { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.next.next = new Node();
lst.next.next.val = 3;
lst.clear();
var emptyLst = new Node();
emptyLst.clear();
```

14.2 操作ログ

14.2.1 lst.clear() に対する操作

- deleteNode: main-new3
- deleteNode: main-new2
- deleteNode: main-new1

14.2.2 emptyLst.clear() に対する操作

- (No explicit deleteNode operations if the initial node is considered the list itself and its next is null)

14.3 手動で復元したコード

14.3.1 lst.clear()

```
this.val = undefined; // Or some default value
this.next = undefined;
```

14.3.2 emptyLst.clear()

```
// No change if already effectively empty
```

14.4 一般化と部分関数の推定

```
// f() = リストを走査してノードを削除する関数
f();
```

14.5 想定の合成された関数

```
clear() {
  // In JavaScript, setting next to null effectively "clears" the list
  // if you assume the current node remains the head.
  // If the head itself should be "emptied", then its val should be reset.
  if (this.next) {
    // Recursively clear the rest of the list
    this.next.clear();
    // Then remove the reference to the cleared tail
    this.next = undefined;
  }
  // Reset the current node's value
  this.val = undefined;
}
```

14.6 oracle の関数

```
clear() {
  if (this.next !== null) {
    this.next.clear();
    this.next = null;
  }
  this.val = undefined; // Or some default value
}
```

15 対象関数: contains(arg)

連結リスト内に指定された値を持つノードが存在するかどうかを返すメソッド。

15.1 初期プログラム

```
class Node {
    contains(arg) { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
lst.next.next = new Node();
lst.next.next.val = 3;
var c1 = lst.contains(0);
var c2 = lst.contains(99);
```

15.2 操作ログ

15.2.1 lst.contains(0) に対する操作

- addVariable: __temp1, true, true, boolean
- addVariable: __temp1, return

15.2.2 lst.contains(99) に対する操作

- addVariable: __temp2, false, true, boolean
- addVariable: __temp2, return

15.3 手動で復元したコード

15.3.1 lst.contains(0)

```
if (this.val === 0) return true;
if (this.next && this.next.val === 0) return true;
return false;
```

15.3.2 lst.contains(99)

```
var current = this;
while (current !== null) {
    if (current.val === 99) return true;
    current = current.next;
}
```

```
}  
return false;
```

15.4 一般化と部分関数の推定

```
// f() = リストを走査してを探し、見つければ、そうでなければを返す関数  
argtruefalse  
return f();
```

15.5 想定の合成された関数

```
contains(arg) {  
  let current = this;  
  while (current !== null) {  
    if (current.val === arg) {  
      return true;  
    }  
    current = current.next;  
  }  
  return false;  
}
```

15.6 oracle の関数

```
contains(arg) {  
  if (this === null) {  
    return false;  
  }  
  if (this.val === arg) {  
    return true;  
  }  
  if (this.next !== null) {  
    return this.next.contains(arg);  
  }  
  return false;  
}
```

16 対象関数: isEmpty()

連結リストが空かどうかを返すメソッド。

16.1 初期プログラム

```
class Node {
    isEmpty() { /* TBD */ }
}
var lst = new Node();
lst.val = 2;
lst.next = new Node();
lst.next.val = 0;
var e1 = lst.isEmpty();
var emptyLst = new Node();
var e2 = emptyLst.isEmpty();
```

16.2 操作ログ

16.2.1 lst.isEmpty() に対する操作

- addVariable: __temp1, false, true, boolean
- addVariable: __temp1, return

16.2.2 emptyLst.isEmpty() に対する操作

- addVariable: __temp2, true, true, boolean
- addVariable: __temp2, return

16.3 手動で復元したコード

16.3.1 lst.isEmpty()

```
return this.val === undefined && this.next === undefined; // Or check for sp
```

16.3.2 emptyLst.isEmpty()

```
return true;
```

16.4 一般化と部分関数の推定

```
// f() = リストが空かどうかを判定する関数
return f();
```

16.5 想定の合成された関数

```
isEmpty() {  
    // Assuming an "empty" list has a null 'next' and potentially an undefined 'va  
    return this.next == null && this.val == undefined;  
}
```

16.6 oracle の関数

```
isEmpty() {  
    return this.next === null && this.val === undefined;  
}
```

17 対象関数: update(oldVal, newVal)

連結リスト内で oldVal を持つすべてのノードの値を newVal に変更するメソッド。

17.1 初期プログラム

```
class Node {  
    update(oldVal, newVal) { /* TBD */ }  
}  
  
var lst = new Node();  
lst.val = 2;  
lst.next = new Node();  
lst.next.val = 0;  
lst.next.next = new Node();  
lst.next.next.val = 3;  
lst.next.next.next = new Node();  
lst.next.next.next.val = 0; // Another 0  
lst.update(0, 99);  
lst.update(5, 100); // Not found
```

17.2 操作ログ

17.2.1 lst.update(0, 99) に対する操作

- addNode: __temp1, 99, true, string
- editEdgeReference: main-new2, main-new2-val, __temp1, val
- addNode: __temp2, 99, true, string
- editEdgeReference: main-new4, main-new4-val, __temp2, val

17.2.2 lst.update(5, 100) に対する操作

- (No operations if value not found)

17.3 手動で復元したコード

17.3.1 lst.update(0, 99)

```
var current = this;
while (current != null) {
    if (current.val === 0) {
        current.val = 99;
    }
    current = current.next;
}
```

17.4 一般化と部分関数の推定

// $f()$ = リストを走査してを探し、に変更する関数 $oldVal newVal$
 $f()$;

17.5 想定の合成された関数

```
update(oldVal, newVal) {
    let current = this;
    while (current != null) {
        if (current.val === oldVal) {
            current.val = newVal; // This implies addNode and editEdgeReference for va
        }
        current = current.next;
    }
}
```

17.6 oracle の関数

```
update(oldVal, newVal) {
    if (this === null) {
        return;
    }
    if (this.val === oldVal) {
        this.val = newVal;
    }
    if (this.next != null) {
        this.next.update(oldVal, newVal);
    }
}
```

```
}  
}
```

18 対象関数: filter(predicate)

指定された条件に一致するノードのみを含む新しい連結リストを返すメソッド。

18.1 初期プログラム

```
class Node {  
  filter(predicate) { /* TBD */ }  
}  
var lst = new Node();  
lst.val = 2;  
lst.next = new Node();  
lst.next.val = 0;  
lst.next.next = new Node();  
lst.next.next.val = 3;  
lst.next.next.next = new Node();  
lst.next.next.next.val = 1;  
var evenList = lst.filter(val => val % 2 === 0);  
var oddList = lst.filter(val => val % 2 !== 0);
```

18.2 操作ログ

18.2.1 lst.filter(val => val % 2 === 0) に対する操作

- addNode: filteredListHead, Node, false, undefined
- addNode: filteredListHeadVal, 2, true, string
- addEdge: filteredListHead, filteredListHeadVal, val
- addNode: filteredListSecond, Node, false, undefined
- addNode: filteredListSecondVal, 0, true, string
- addEdge: filteredListSecond, filteredListSecondVal, val
- addEdge: filteredListHead, filteredListSecond, next
- addVariable: filteredListHead, return

18.2.2 `lst.filter(val => val % 2 !== 0)` に対する操作

- `addNode: oddListHead, Node, false, undefined`
- `addNode: oddListHeadVal, 3, true, string`
- `addEdge: oddListHead, oddListHeadVal, val`
- `addNode: oddListSecond, Node, false, undefined`
- `addNode: oddListSecondVal, 1, true, string`
- `addEdge: oddListSecond, oddListSecondVal, val`
- `addEdge: oddListHead, oddListSecond, next`
- `addVariable: oddListHead, return`

18.3 手動で復元したコード

18.3.1 `lst.filter(val => val % 2 === 0)`

```
var newList = null;
var tail = null;
var current = this;
while (current !== null) {
  if (current.val % 2 === 0) {
    var newNode = new Node();
    newNode.val = current.val;
    if (newList === null) {
      newList = newNode;
      tail = newNode;
    } else {
      tail.next = newNode;
      tail = newNode;
    }
  }
  current = current.next;
}
return newList;
```

18.4 一般化と部分関数の推定

```
// f() = に基づいて新しいリストを構築する関数 predicate
return f();
```

18.5 想定の合成された関数

```
filter(predicate) {
    let newList = null;
    let currentNewNode = null;
    let currentOriginalNode = this;

    while (currentOriginalNode !== null) {
        if (predicate(currentOriginalNode.val)) {
            const newNode = new Node();
            newNode.val = currentOriginalNode.val;
            if (newList === null) {
                newList = newNode;
                currentNewNode = newNode;
            } else {
                currentNewNode.next = newNode;
                currentNewNode = newNode;
            }
        }
        currentOriginalNode = currentOriginalNode.next;
    }
    return newList;
}
```

18.6 oracle の関数

```
filter(predicate) {
    if (this === null) {
        return null;
    }
    if (predicate(this.val)) {
        const newNode = new Node();
        newNode.val = this.val;
        newNode.next = this.next ? this.next.filter(predicate) : null;
        return newNode;
    } else {
        return this.next ? this.next.filter(predicate) : null;
    }
}
```

19 対象関数: deduplicate()

連結リストから重複する値を削除し、一意の値を保持するメソッド。

19.1 初期プログラム

```
class Node {
    deduplicate() { /* TBD */ }
}

var lst1 = new Node();
lst1.val = 1;
lst1.next = new Node();
lst1.next.val = 2;
lst1.next.next = new Node();
lst1.next.next.val = 1; // Duplicate
lst1.next.next.next = new Node();
lst1.next.next.next.val = 3;
lst1.deduplicate();

var lst2 = new Node();
lst2.val = 1;
lst2.next = new Node();
lst2.next.val = 1;
lst2.next.next = new Node();
lst2.next.next.val = 1; // Multiple duplicates
lst2.deduplicate();
```

19.2 操作ログ

19.2.1 lst1.deduplicate() に対する操作

- deleteNode: main-new3
- editEdgeReference: main-new2, main-new3, main-new4, next

19.2.2 lst2.deduplicate() に対する操作

- deleteNode: main-new6
- editEdgeReference: main-new5, main-new6, main-new7, next
- deleteNode: main-new7
- editEdgeReference: main-new5, main-new7, main-new8, next

19.3 手動で復元したコード

19.3.1 lst1.deduplicate()

```
var current = this;
var seen = new Set();
if (current) {
```

```

    seen.add(current.val);
}
while (current && current.next) {
    if (seen.has(current.next.val)) {
        current.next = current.next.next;
    } else {
        seen.add(current.next.val);
        current = current.next;
    }
}
}

```

19.4 一般化と部分関数の推定

// $f()$ = リストを走査し、重複を検出し、ノードを削除する関数
 $f()$;

19.5 想定の合成された関数

```

deduplicate() {
    let seen = new Set();
    let current = this;
    let prev = null;

    while (current !== null) {
        if (seen.has(current.val)) {
            // Delete the current node
            if (prev !== null) {
                prev.next = current.next; // editEdgeReference
            } else {
                // If current is head and is a duplicate, the head shifts
                // This case is complex as 'this' itself would need to change.
                // For simplicity, assuming 'this' is not removed directly,
                // or handled by a return value.
            }
            // Implicit deleteNode for current
        } else {
            seen.add(current.val);
            prev = current;
        }
        current = current.next;
    }
}

```

19.6 oracle の関数

```
deduplicate() {
  let seen = new Set();
  let current = this;
  let prev = null;

  while (current !== null) {
    if (seen.has(current.val)) {
      // Skip current node (effectively deleting it)
      if (prev !== null) {
        prev.next = current.next;
      } else {
        // If head is a duplicate, advance the head (caller responsibility)
        // This requires 'return this.next' logic
      }
    } else {
      seen.add(current.val);
      prev = current;
    }
    current = current.next;
  }
}
\end{lstlisting}
```

% \clearpage

\section{対象関数}{: \texttt{reverse()}}連結リストのノードの順序を反転させるメソッド。

\subsection{初期プログラム}{}

```
\begin{lstlisting}[language=JavaScript]
class Node {
  reverse() { /* TBD */ }
}
var lst = new Node();
lst.val = 1;
lst.next = new Node();
lst.next.val = 2;
lst.next.next = new Node();
lst.next.next.val = 3;
lst.reverse();
\end{lstlisting}
```

19.7 操作ログ

19.7.1 lst.reverse() に対する操作

- editEdgeReference: main-new3, null, main-new2, next
- editEdgeReference: main-new2, main-new3, main-new1, next
- editEdgeReference: main-new1, main-new2, null, next
- addVariable: main-new3, return

19.8 手動で復元したコード

19.8.1 lst.reverse()

```
var prev = null;
var current = this;
while (current !== null) {
  var nextTemp = current.next;
  current.next = prev;
  prev = current;
  current = nextTemp;
}
return prev;
\end{lstlisting}
```

\subsection{一般化と部分関数の推定{}}

```
\begin{lstlisting}[language=JavaScript]
// f() = ポインタを操作してリストを反転させる関数
return f();
```

19.9 想定 of 合成された関数

```
reverse() {
  let prev = null;
  let current = this;
  let next = null;

  while (current !== null) {
    next = current.next; // Store next node
    current.next = prev; // Reverse current node's next pointer (editEdgeReferen
    prev = current; // Move prev to current node
    current = next; // Move current to next node
  }
  // At the end, prev will be the new head
```

```
    return prev;
}
```

19.10 oracle の関数

```
reverse(head = this, prev = null) {
    if (head === null) {
        return prev;
    }
    const nextNode = head.next;
    head.next = prev;
    return this.reverse(nextNode, head);
}
```

20 対象関数: sort()

連結リストのノードを値に基づいてソートするメソッド。

20.1 初期プログラム

```
class Node {
    sort() { /* TBD */ }
}
var lst = new Node();
lst.val = 3;
lst.next = new Node();
lst.next.val = 1;
lst.next.next = new Node();
lst.next.next.val = 2;
lst.sort();
```

20.2 操作ログ

20.2.1 lst.sort() に対する操作

- addNode: __temp1, 1, true, string
- editEdgeReference: main-new1, main-new1-val, __temp1, val
- addNode: __temp2, 2, true, string
- editEdgeReference: main-new2, main-new2-val, __temp2, val
- addNode: __temp3, 3, true, string
- editEdgeReference: main-new3, main-new3-val, __temp3, val

20.3 手動で復元したコード

20.3.1 lst.sort()

```
// Example: Bubble sort on values
var swapped;
do {
    swapped = false;
    var current = this;
    while (current !== null && current.next !== null) {
        if (current.val > current.next.val) {
            var temp = current.val;
            current.val = current.next.val;
            current.next.val = temp;
            swapped = true;
        }
        current = current.next;
    }
} while (swapped);
```

20.4 一般化と部分関数の推定

```
// f() = ソートアルゴリズムを実装し、ノードの値を交換する関数
f();
```

20.5 想定の合成された関数

```
sort() {
    // Simple bubble sort (modifying values directly)
    let swapped;
    do {
        swapped = false;
        let current = this;
        while (current !== null && current.next !== null) {
            if (current.val > current.next.val) {
                // Swap values (implies addNode for temp val, editEdgeReference for val)
                let temp = current.val;
                current.val = current.next.val;
                current.next.val = temp;
                swapped = true;
            }
            current = current.next;
        }
    } while (swapped);
}
```

20.6 oracle の関数

```
sort() {
    // For simplicity, converting to array, sorting, then back to list.
    // A true in-place linked list sort would be more complex and recursive.
    const arr = [];
    let current = this;
    while (current !== null) {
        arr.push(current.val);
        current = current.next;
    }
    arr.sort((a, b) => a - b);

    current = this;
    let i = 0;
    while (current !== null) {
        current.val = arr[i++];
        current = current.next;
    }
}
```

21 対象関数: splitAt(i)

連結リストを*i*番目のノードで2つのリストに分割するメソッド。元のリストは*i*番目のノードまでとなり、*i*番目以降は新しいリストとなる。

21.1 初期プログラム

```
class Node {
    splitAt(i) { /* TBD */ }
}

var lst = new Node();
lst.val = 1;
lst.next = new Node();
lst.next.val = 2;
lst.next.next = new Node();
lst.next.next.val = 3;
lst.next.next.next = new Node();
lst.next.next.next.val = 4;
var secondList = lst.splitAt(1);
var thirdList = lst.splitAt(0);
```

21.2 操作ログ

21.2.1 lst.splitAt(1) に対する操作

- editEdgeReference: main-new2, main-new3, null, next
- addVariable: main-new3, return

21.2.2 lst.splitAt(0) に対する操作

- editEdgeReference: main-new1, main-new2, null, next
- addVariable: main-new2, return

21.3 手動で復元したコード

21.3.1 lst.splitAt(1)

```
var current = this;
var count = 0;
while (current !== null && count < 1) {
    current = current.next;
    count++;
}
if (current && current.next) {
    var newHead = current.next;
    current.next = null;
    return newHead;
}
return null;
```

21.4 一般化と部分関数の推定

```
// f() = 指定されたインデックスまでリストを走査し、エッジを操作して分割する関数
return f();
```

21.5 想定合成された関数

```
splitAt(i) {
    if (i < 0) return null; // Or throw error
    let current = this;
    let count = 0;

    // Handle splitting at index 0 (return the whole list, make original head null)
    if (i === 0) {
```

```

    const fullList = this;
    // How to represent 'this' becoming null or a new empty node in Kanon?
    // Maybe the 'addVariable: null, return' for the original list is enough.
    return fullList;
}

while (current !== null && count < i - 1) {
    current = current.next;
    count++;
}

if (current !== null && current.next !== null) {
    const newHead = current.next;
    current.next = null; // editEdgeReference: current, newHead, null, next
    return newHead;
}
return null; // No split possible
}

```

21.6 oracle の関数

```

splitAt(i) {
    function find(node, index) {
        if (node === null || index < 0) {
            return null;
        }
        if (index === 0) {
            return node;
        }
        return find(node.next, index - 1);
    }

    if (i === 0) {
        const originalList = this.next; // Assuming current 'this' becomes isolated
        this.next = null;
        return originalList;
    }

    const nodeBeforeSplit = find(this, i - 1);
    if (nodeBeforeSplit !== null && nodeBeforeSplit.next !== null) {
        const newHead = nodeBeforeSplit.next;
        nodeBeforeSplit.next = null;
        return newHead;
    }
    return null;
}

```

```
}
```

22 対象関数: sublist(startIndex, endIndex)

連結リストの指定された範囲のノードを含む新しい連結リストを作成し返すメソッド。

22.1 初期プログラム

```
class Node {
  sublist(startIndex, endIndex) { /* TBD */ }
}
var lst = new Node();
lst.val = 1;
lst.next = new Node();
lst.next.val = 2;
lst.next.next = new Node();
lst.next.next.val = 3;
lst.next.next.next = new Node();
lst.next.next.next.val = 4;
var sub1 = lst.sublist(0, 1); // 1, 2
var sub2 = lst.sublist(2, 3); // 3, 4
var sub3 = lst.sublist(0, 0); // 1
```

22.2 操作ログ

22.2.1 lst.sublist(0, 1) に対する操作

- addNode: newSublistHead, Node, false, undefined
- addNode: newSublistHeadVal, 1, true, string
- addEdge: newSublistHead, newSublistHeadVal, val
- addNode: newSublistSecond, Node, false, undefined
- addNode: newSublistSecondVal, 2, true, string
- addEdge: newSublistSecond, newSublistSecondVal, val
- addEdge: newSublistHead, newSublistSecond, next
- addVariable: newSublistHead, return

22.2.2 lst.sublist(2, 3) に対する操作

- addNode: newSublistHead2, Node, false, undefined
- addNode: newSublistHead2Val, 3, true, string
- addEdge: newSublistHead2, newSublistHead2Val, val
- addNode: newSublistSecond2, Node, false, undefined
- addNode: newSublistSecond2Val, 4, true, string
- addEdge: newSublistSecond2, newSublistSecond2Val, val
- addEdge: newSublistHead2, newSublistSecond2, next
- addVariable: newSublistHead2, return

22.3 手動で復元したコード

22.3.1 lst.sublist(0, 1)

```
var newList = new Node();
newList.val = this.val;
newList.next = new Node();
newList.next.val = this.next.val;
return newList;
```

22.4 一般化と部分関数の推定

```
// f() = 指定された範囲のノードを複製して新しいリストを作成する関数
return f();
```

22.5 想定的合成された関数

```
sublist(startIndex, endIndex) {
  if (startIndex < 0 || startIndex > endIndex) return null; // Invalid range
  let newList = null;
  let currentNewNode = null;
  let currentOriginalNode = this;
  let currentIndex = 0;

  // Advance to startIndex
  while (currentOriginalNode !== null && currentIndex < startIndex) {
    currentOriginalNode = currentOriginalNode.next;
    currentIndex++;
  }
}
```

```

// Copy nodes within the range
while (currentOriginalNode !== null && currentIndex <= endIndex) {
  const newNode = new Node();
  newNode.val = currentOriginalNode.val; // addNode for newNode, addNode for v

  if (newList === null) {
    newList = newNode;
    currentNewNode = newNode;
  } else {
    currentNewNode.next = newNode; // addEdge for next
    currentNewNode = newNode;
  }
  currentOriginalNode = currentOriginalNode.next;
  currentIndex++;
}
return newList;
}

```

22.6 oracle の関数

```

sublist(startIndex, endIndex) {
  if (startIndex < 0 || startIndex > endIndex) {
    return null;
  }
  let newList = null;
  let tail = null;
  let current = this;
  let currentIndex = 0;

  while (current !== null && currentIndex <= endIndex) {
    if (currentIndex >= startIndex) {
      const newNode = new Node();
      newNode.val = current.val;
      if (newList === null) {
        newList = newNode;
        tail = newNode;
      } else {
        tail.next = newNode;
        tail = newNode;
      }
    }
    current = current.next;
    currentIndex++;
  }
}

```

```
    return newList;
}
```

23 対象関数: removeOdds()

連結リストから偶数番目のノードを削除するメソッド。

23.1 初期プログラム

```
class Node {
    removeOdds() { /* TBD */ }
}
var lst = new Node();
lst.val = 1; // Index 0 (keep)
lst.next = new Node();
lst.next.val = 2; // Index 1 (remove)
lst.next.next = new Node();
lst.next.next.val = 3; // Index 2 (keep)
lst.next.next.next = new Node();
lst.next.next.next.val = 4; // Index 3 (remove)
lst.removeOdds();
```

23.2 操作ログ

23.2.1 lst.removeOdds() に対する操作

- deleteNode: main-new2
- editEdgeReference: main-new1, main-new2, main-new3, next
- deleteNode: main-new4
- editEdgeReference: main-new3, main-new4, null, next

23.3 手動で復元したコード

23.3.1 lst.removeOdds()

```
var current = this;
var index = 0;
while (current !== null && current.next !== null) {
    if (index % 2 === 1) { // Check if the next node is odd-indexed (0-based)
        // This logic removes the current node if it's odd.
        // If aiming to remove the node at odd index, adjust.
        // Assuming removing the NEXT node if it's odd-indexed
        current.next = current.next.next;
    }
    current = current.next;
    index++;
}
```

```

        // deleteNode for current.next
    } else {
        current = current.next;
    }
    index++;
}

```

23.4 一般化と部分関数の推定

// $f()$ = リストを走査し、偶数番目のノードを削除する関数
 $f()$;

23.5 想定の合成された関数

```

removeOdds() {
    if (this === null) return;
    let current = this;
    let index = 0;

    // Handle the case where the head itself needs to be removed (if it's at an odd
    // Assuming 0-indexed: remove nodes at index 1, 3, 5...
    while (current !== null && current.next !== null) {
        if (index % 2 === 0) { // If current node is at an even index
            current.next = current.next.next; // Skip the next (odd-indexed) node (edit)
            // current remains the same, as its next has changed
        } else {
            current = current.next; // Move to the next node
        }
        index++;
    }
}

```

23.6 oracle の関数

```

removeOdds() {
    if (this === null || this.next === null) {
        return;
    }
    // Recursive approach:
    // 'this' is at index 0 (even), so we keep it.
    // We then call removeOdds on this.next to handle the rest of the list.
    // If this.next was at index 1 (odd) and needs to be removed,
    // this.next will be replaced with this.next.next from the recursive call.
}

```

```

    this.next = this.next.removeOddsRecursive(1); // Helper to handle indices
}

// Helper function that explicitly takes the current index
removeOddsRecursive(currentIndex) {
    if (this === null) {
        return null;
    }
    if (currentIndex % 2 === 1) { // If current node is at an odd index
        // This node should be removed. Return its next node.
        // The caller will then link to this.next.
        return this.next ? this.next.removeOddsRecursive(currentIndex + 1) : null;
    } else {
        // This node should be kept. Link its next to the result of recursive call
        this.next = this.next ? this.next.removeOddsRecursive(currentIndex + 1) : null;
        return this;
    }
}
}

```

24 対象関数: pad(length, value)

連結リストを指定された length になるまで value でパディングするメソッド。

24.1 初期プログラム

```

class Node {
    pad(length, value) { /* TBD */ }
}

var lst = new Node();
lst.val = 1;
lst.next = new Node();
lst.next.val = 2;
lst.pad(5, 99); // Pad to length 5 with 99s
var shortLst = new Node();
shortLst.val = 1;
shortLst.pad(1, 0); // No padding needed

```

24.2 操作ログ

24.2.1 lst.pad(5, 99) に対する操作

- addNode: temp1, Node, false, undefined
- addNode: temp2, 99, true, string

- addEdge: temp1, temp2, val
- editEdgeReference: main-new2, null, temp1, next
- addNode: temp3, Node, false, undefined
- addNode: temp4, 99, true, string
- addEdge: temp3, temp4, val
- addEdge: temp1, temp3, next
- addNode: temp5, Node, false, undefined
- addNode: temp6, 99, true, string
- addEdge: temp5, temp6, val
- addEdge: temp3, temp5, next

24.2.2 shortLst.pad(1, 0) に対する操作

- (No operations)

24.3 手動で復元したコード

24.3.1 lst.pad(5, 99)

```
var current = this;
var count = 0;
while (current.next !== null) {
  current = current.next;
  count++;
}
count++; // Count current node

while (count < 5) {
  var newNode = new Node();
  newNode.val = 99;
  current.next = newNode;
  current = newNode;
  count++;
}
```

24.4 一般化と部分関数の推定

// $f()$ = リストの長さを計算し、不足しているノードを追加する関数
 $f()$;

24.5 想定の合成された関数

```
pad(length, value) {
  let current = this;
  let count = 0;

  // Find the current length
  while (current !== null && current.next !== null) {
    current = current.next;
    count++;
  }
  // Add 1 for the head node itself
  count++;

  // Add padding nodes
  while (count < length) {
    const newNode = new Node();
    newNode.val = value; // addNode for newNode, addNode for val, addEdge for val
    current.next = newNode; // addEdge for next
    current = newNode;
    count++;
  }
}
```

24.6 oracle の関数

```
pad(length, value) {
  let currentLength = 0;
  let current = this;
  while (current !== null) {
    currentLength++;
    if (current.next === null) {
      break; // Found the tail
    }
    current = current.next;
  }

  while (currentLength < length) {
    const newNode = new Node();
    newNode.val = value;
    if (current === null) { // For an empty list
      // This scenario is tricky with 'this' as context.
      // Assuming 'this' is always a valid node even if it represents an empty list
      // and new nodes are appended to it.
      // If 'this' is the only node and its val is undefined, set its val to value
    }
  }
}
```

```

        // For simplicity, we assume this.next is where new nodes are added.
    } else {
        current.next = newNode;
        current = newNode;
    }
    currentLength++;
}
}

```

25 対象関数: merge(otherList)

別の連結リストを現在のリストと結合し、ソートされた新しい連結リストを返すメソッド（破壊的ではない場合）。

25.1 初期プログラム

```

class Node {
    merge(otherList) { /* TBD */ }
}
var lst1 = new Node();
lst1.val = 1;
lst1.next = new Node();
lst1.next.val = 3;
var lst2 = new Node();
lst2.val = 2;
lst2.next = new Node();
lst2.next.val = 4;
var mergedList = lst1.merge(lst2);

```

25.2 操作ログ

25.2.1 lst1.merge(lst2) に対する操作

- addNode: mergedHead, Node, false, undefined
- addNode: mergedHeadVal, 1, true, string
- addEdge: mergedHead, mergedHeadVal, val
- addNode: mergedSecond, Node, false, undefined
- addNode: mergedSecondVal, 2, true, string
- addEdge: mergedSecond, mergedSecondVal, val
- addEdge: mergedHead, mergedSecond, next

- addNode: mergedThird, Node, false, undefined
- addNode: mergedThirdVal, 3, true, string
- addEdge: mergedThird, mergedThirdVal, val
- addEdge: mergedSecond, mergedThird, next
- addNode: mergedFourth, Node, false, undefined
- addNode: mergedFourthVal, 4, true, string
- addEdge: mergedFourth, mergedFourthVal, val
- addEdge: mergedThird, mergedFourth, next
- addVariable: mergedHead, return

25.3 手動で復元したコード

25.3.1 lst1.merge(lst2)

```

var mergedHead = new Node();
var currentMerged = mergedHead;
var current1 = this;
var current2 = otherList;

while (current1 != null && current2 != null) {
    var newNode = new Node();
    if (current1.val <= current2.val) {
        newNode.val = current1.val;
        current1 = current1.next;
    } else {
        newNode.val = current2.val;
        current2 = current2.next;
    }
    currentMerged.next = newNode;
    currentMerged = newNode;
}

while (current1 != null) {
    var newNode = new Node();
    newNode.val = current1.val;
    currentMerged.next = newNode;
    currentMerged = newNode;
    current1 = current1.next;
}

while (current2 != null) {

```

```

    var newNode = new Node();
    newNode.val = current2.val;
    currentMerged.next = newNode;
    currentMerged = newNode;
    current2 = current2.next;
}
return mergedHead.next; // Skip dummy head

```

25.4 一般化と部分関数の推定

// $f()$ = つのソート済みリストをマージして新しいソート済みリストを構築する関数 2

```

return f();

```

25.5 想定的合成された関数

```

merge(otherList) {
    let mergedHead = null;
    let currentMerged = null;
    let current1 = this;
    let current2 = otherList;

    while (current1 !== null && current2 !== null) {
        let newNode = new Node();
        // Add newNode, its val, and edge to val
        if (current1.val <= current2.val) {
            newNode.val = current1.val;
            current1 = current1.next;
        } else {
            newNode.val = current2.val;
            current2 = current2.next;
        }

        if (mergedHead === null) {
            mergedHead = newNode;
            currentMerged = newNode;
        } else {
            currentMerged.next = newNode; // addEdge for next
            currentMerged = newNode;
        }
    }

    // Append remaining nodes
    let remaining = current1 !== null ? current1 : current2;

```

```

while (remaining !== null) {
  let newNode = new Node();
  newNode.val = remaining.val;
  if (mergedHead === null) {
    mergedHead = newNode;
    currentMerged = newNode;
  } else {
    currentMerged.next = newNode;
    currentMerged = newNode;
  }
  remaining = remaining.next;
}
return mergedHead;
}

```

25.6 oracle の関数

```

merge(otherList) {
  if (this === null) return otherList;
  if (otherList === null) return this;

  let resultHead;
  if (this.val <= otherList.val) {
    resultHead = this;
    resultHead.next = this.next ? this.next.merge(otherList) : otherList;
  } else {
    resultHead = otherList;
    resultHead.next = otherList.next ? this.merge(otherList.next) : this;
  }
  return resultHead;
}

```

26 対象関数: compact()

連結リストから null または undefined の値を削除するメソッド。

26.1 初期プログラム

```

class Node {
  compact() { /* TBD */ }
}
var lst = new Node();
lst.val = 1;

```

```

lst.next = new Node();
lst.next.val = undefined; // Null value
lst.next.next = new Node();
lst.next.next.val = 2;
lst.next.next.next = new Node();
lst.next.next.next.val = null; // Another null value
lst.compact();

```

26.2 操作ログ

26.2.1 lst.compact() に対する操作

- deleteNode: main-new2
- editEdgeReference: main-new1, main-new2, main-new3, next
- deleteNode: main-new4
- editEdgeReference: main-new3, main-new4, null, next

26.3 手動で復元したコード

26.3.1 lst.compact()

```

var current = this;
while (current !== null && current.next !== null) {
  if (current.next.val === null || current.next.val === undefined) {
    current.next = current.next.next; // Skip the null/undefined node
  } else {
    current = current.next;
  }
}
// Handle head if it's null/undefined (requires return value)

```

26.4 一般化と部分関数の推定

```

// f() = リストを走査し、null/ノードを削除する関数 undefined
f();

```

26.5 想定合成された関数

```

compact() {
  // Handle head node if it's null/undefined
  // This typically means the function needs to return the new head
  // For simplicity, if 'this' is null/undefined, it might be implicitly handled
}

```

```

// Assuming 'this' is always a valid starting node.
if (this === null) return null;

let current = this;
while (current !== null && current.next !== null) {
  if (current.next.val === null || current.next.val === undefined) {
    // Delete current.next (editEdgeReference to skip, deleteNode)
    current.next = current.next.next;
  } else {
    current = current.next;
  }
}
// If the original head was null/undefined, it also needs to be handled
// For in-place modification of 'this', this is tricky.
// If the intention is to modify the list pointed to by 'this'
// and return the potentially new head.
if (this.val === null || this.val === undefined) {
  return this.next ? this.next.compact() : null; // Recursively compact from n
}
return this;
}

```

26.6 oracle の関数

```

compact() {
  if (this === null) {
    return null;
  }
  // If current node's value is null/undefined, return its next (effectively
  if (this.val === null || this.val === undefined) {
    return this.next ? this.next.compact() : null;
  }
  // Otherwise, keep current node and compact the rest of the list
  this.next = this.next ? this.next.compact() : null;
  return this;
}

```